



An-Najah National University

Faculty of Engineering and Information Technology Department of Computer Science

Prompt Quality Evaluator

Using Instruction-Tuned Large Language Models

Submitted to : Dr. Hamed Abdelhaq

Submitted by : Mayar Basheer ,Omar El Haj Daoud ,Yousef Dabeek

Abstract

This project presents a Prompt Quality Evaluator that uses an instruction-tuned Large Language Model to assess the quality of user-written prompts. The system receives a structured input consisting of a task, reference prompt, submitted prompt, and rubric, then produces a discrete score from 0 to 4 with a natural-language rationale. A custom prompt evaluation dataset was created, refined, formatted for instruction tuning, and split into balanced train, validation, and test sets. The base model, Mistral-7B-Instruct-v0.2, was evaluated before fine-tuning and then adapted using LoRA with the Unsloth framework. On the held-out test set, baseline accuracy was 56.39%, while the fine-tuned model achieved 96.00% accuracy. Mean Absolute Error decreased from 0.5013 to 0.0425, and Quadratic Weighted Kappa improved from 0.8490 to 0.9880, demonstrating strong alignment with the rubric-based labels.

Table of Contents

1. Introduction
2. Dataset
3. Methodology
4. Experimental Setup
5. Results
6. Discussion
7. Recommendations and Future Work
8. Team Contributions
9. Conclusion
10. References

1. Introduction

1.1 Motivation

Large Language Models (LLMs) have become widely used in education, software development, writing, business communication, and research. However, the quality of an LLM response depends heavily on the quality of the prompt given by the user. A vague prompt usually produces vague output, while a clear and specific prompt can guide the model toward a more useful response. For students and non-expert users, writing effective prompts is often difficult because they may not know how to include context, constraints, audience, tone, or desired format.

This project addresses that problem by building an automatic Prompt Quality Evaluator. Instead of only generating responses, the system evaluates how good a submitted prompt is. It assigns a numerical quality score and explains the reasoning behind that score. This makes the system useful not only for prediction, but also for learning and feedback.

1.2 Problem Statement

The goal of the project is to fine-tune an instruction-tuned LLM so that it can grade prompts according to a rubric. The model must take structured input that includes a task, a reference prompt, a submitted prompt, and rubric criteria. It must then output a score from 0 to 4 and a rationale explaining the evaluation. The score should reflect clarity, specificity, useful constraints, and completeness.

1.3 Objectives

- Build a custom dataset for rubric-based prompt quality evaluation.
- Represent every example using task, reference, submission, rubric, score, and rationale fields.
- Evaluate the base Mistral-7B-Instruct model on the held-out test set.
- Fine-tune the model using LoRA and Unsloth.
- Compare baseline and fine-tuned performance using Accuracy, MAE, and QWK.
- Analyze the improvements and limitations of the fine-tuned evaluator.

2. Dataset

2.1 Dataset Construction

The project uses a custom prompt evaluation dataset rather than a public dataset because the selected task requires examples that explicitly connect a submitted prompt to a rubric-based score and rationale. The dataset was created using synthetic generation and manual refinement. The synthetic approach made it possible to cover many prompt-writing scenarios, while manual review and cleaning were used to reduce repetition, separate weak scores more clearly, and improve rationale quality.

The original candidate dataset contained 15,000 balanced examples. For this project, a processed subset containing 1,350 formatted examples was used as the initial dataset. To increase dataset diversity and size, an additional 926 examples generated through the Claude API and 400 examples collected through Claude chat batches were incorporated. After merging, cleaning, deduplication, and balancing, the final dataset contained 2,620 examples. The dataset was then split into 1,830 training examples, 390 validation examples, and 400 test examples. The held-out test set is balanced, with 80 examples for each score from 0 to 4. This prevents the evaluation metrics from being dominated by any single class.

2.2 Dataset Fields

Field	Description
task	The task or domain for which the user is writing a prompt.
reference	A high-quality reference prompt that represents the desired standard.
submission	The candidate prompt being evaluated.
rubric	The criteria used to evaluate the submitted prompt.
score	A discrete label from 0 to 4.
rationale	A natural-language explanation justifying the score.

2.3 Score Scale

Score	Meaning	Description
0	Unusable	The prompt is irrelevant, unclear, or cannot be evaluated.
1	Poor	The prompt is related to the task but extremely vague.
2	Acceptable	The prompt states the task but misses important details or constraints.
3	Good	The prompt is mostly clear and useful but can still be improved.
4	Excellent	The prompt is clear, specific, complete, and includes useful constraints.

2.4 Dataset Statistics

Data Source	Samples	Percentage
Original Processed Dataset	1,350	50.4%
Claude API Dataset	926	34.6%
Chat Batches (4 × 100)	400	14.9%
Total Before Cleaning	2,676	100%
Duplicates Removed	0	—
Final Balanced Dataset	2,620	—

2.5 Preprocessing and Quality Assurance

Several preprocessing and quality assurance steps were applied before training and evaluation. The dataset was converted into an instruction-tuning format and checked for class distribution, duplicates, and potential leakage between train, validation, and test splits. Earlier dataset versions contained overlapping and conflicting examples, particularly between score 0 and score 1, which were manually refined to create clearer class boundaries. After merging the datasets, class balancing was performed to ensure equal representation across all five score levels. The final balanced dataset contained 2,620 samples (524 per class) and was split into 1,830 training, 390 validation, and 400 test examples. Leakage checks confirmed zero overlap between all splits, ensuring a fair and reliable evaluation process.

Key quality assurance measures included:

- Balanced label distribution across all score classes.
- Structured instruction–input–output formatting.
- Deduplication and leakage detection..
- Consistent score definitions from 0 to 4.

3. Methodology

3.1 Model Choice

The base model used in this project is Mistral-7B-Instruct-v0.2. It is an instruction-tuned LLM capable of following structured prompts and generating detailed textual responses. The model was first evaluated without fine-tuning to establish a baseline. Then, it was fine-tuned using LoRA on the prepared prompt evaluation dataset.

3.2 Instruction-Tuning Format

Each dataset example was converted into an instruction-tuning format. The instruction asks the model to evaluate the quality of a submitted prompt based on the rubric. The input contains the task, reference prompt, submitted prompt, and rubric. The expected output contains a score and rationale. This format teaches the model not only to classify, but also to explain its decision.

Component	Example Role
Instruction	Evaluate the quality of the submitted prompt based on the rubric.
Input	Task, reference prompt, submitted prompt, and rubric.
Output	Score: 0-4 and a rationale.

3.3 LoRA Fine-Tuning

Full fine-tuning of a 7B-parameter model is computationally expensive. Therefore, this project used LoRA, a parameter-efficient fine-tuning method that trains small adapter matrices while keeping most of the original model weights frozen. This reduces GPU memory requirements and training time while still allowing the model to adapt to the specialized task.

The Unsloth framework was used to support efficient fine-tuning. This choice is appropriate for a course project because it enables practical experimentation with large instruction-tuned models on available GPU resources.

3.4 System Pipeline

- Generate and refine prompt quality datasets.
- Merge, clean, and balance the collected datasets.
- Format examples for instruction tuning.
- Split into train, validation, and test sets.
- Run baseline inference using the base model.
- Fine-tune the model using LoRA and Unsloth.

- Run fine-tuned inference on the same test set.
- Compute metrics and analyze results.

4. Experimental Setup

The experiments were conducted using Python and several machine learning libraries. Hugging Face Transformers was used for model loading and inference, PEFT and LoRA were used for parameter-efficient fine-tuning, Unsloth was used to optimize training efficiency and memory usage, and scikit-learn was used to compute evaluation metrics. GPU resources were utilized to perform both inference and fine-tuning efficiently.

Item	Value
Base model	Mistral-7B-Instruct-v0.2
Fine-tuning method	LoRA using Unsloth
Score range	0 to 4
Evaluation set	400 examples
Metrics	Accuracy, MAE, QWK
Prediction files	baseline_predictions.json and finetuned_predictions.json

Training Epochs	3
Training Samples	1,830
Validation Samples	390
GPU	NVIDIA A100 80GB
Framework	Unsloth + HuggingFace Transformers

5. Results

5.1 Overall Metrics

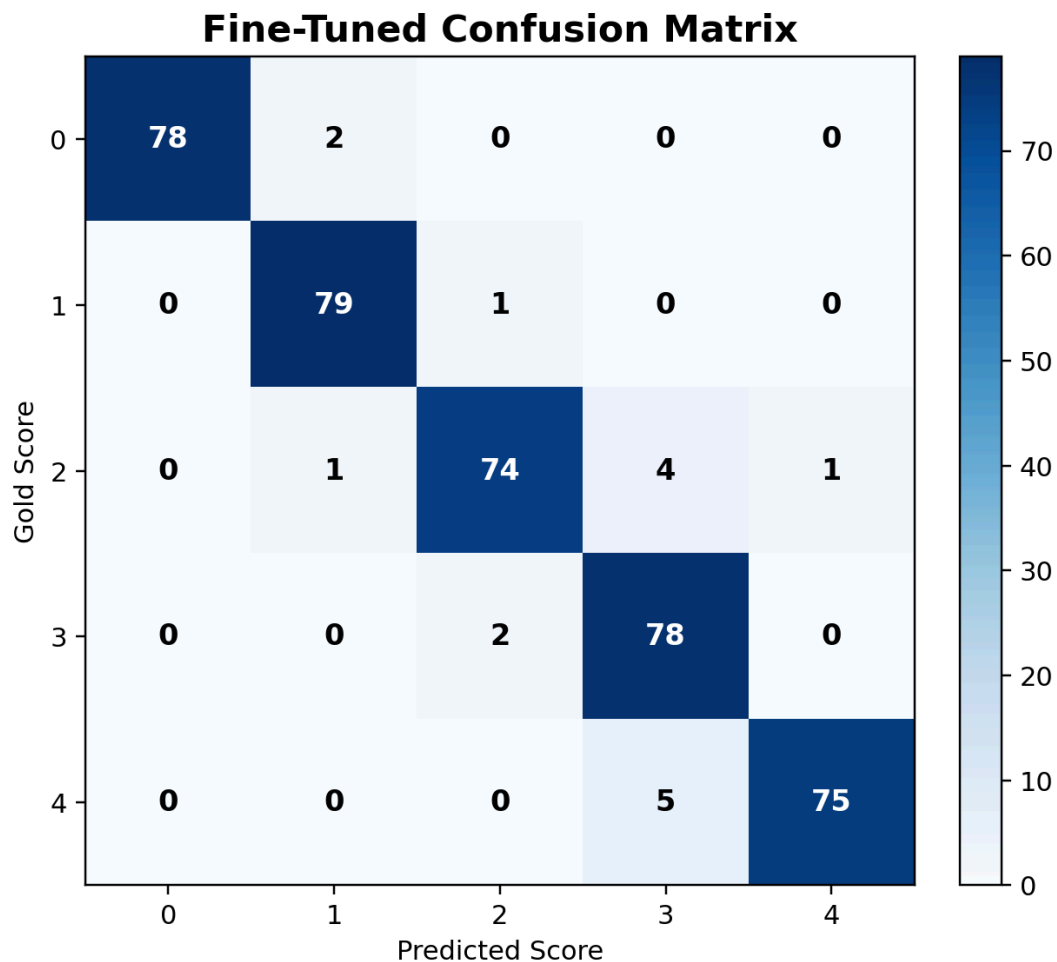
The fine-tuned model substantially outperformed the baseline across all evaluation metrics. Accuracy improved from 56.39% to 96.00%, while Mean Absolute Error (MAE) decreased from 0.5013 to 0.0425, indicating significantly smaller prediction errors. Quadratic Weighted Kappa (QWK) increased from 0.8490 to 0.9880, demonstrating near-perfect agreement between the model predictions and the ground-truth ordinal labels. These results indicate that LoRA fine-tuning successfully adapted the base Mistral-7B-Instruct-v0.2 model to the rubric-based prompt quality evaluation task.

Metric	Baseline (Zero-Shot)	Fine-Tuned (LoRA)	Improvement
Accuracy	56.39%	96.00%	+39.61 pp
MAE	0.501	0.0425	-91.5%
QWK	0.849	0.988	+0.139

5.2 Confusion Matrix Analysis

The baseline confusion matrix reveals that the base model struggled primarily with the middle score categories (1, 2, and 3). While score 0 was classified correctly in most cases (76 out of 80 examples), score 2 exhibited substantial confusion, with many examples being predicted as score 3. Similarly, score 3 and score 4 were frequently confused with one another. Most classification errors occurred between neighboring score levels rather than distant categories, indicating that the model had a reasonable understanding of prompt quality but lacked precise alignment with the project rubric. This explains why the Quadratic Weighted Kappa (QWK) score remained relatively high (0.849) despite the lower exact-match accuracy of 56.39%.

After fine-tuning, the confusion matrix became much more concentrated along the diagonal. Most predictions matched the correct score exactly. The remaining errors were mostly near misses, such as confusing score 3 with score 4 or score 2 with score 3. This indicates that the fine-tuned model learned the scoring scale effectively.



6. Discussion

The results demonstrate that LoRA fine-tuning successfully adapted Mistral-7B-Instruct-v0.2 to the rubric-based prompt quality evaluation task. While the baseline model showed a reasonable understanding of prompt quality (QWK = 0.849), its exact classification performance was limited (Accuracy = 56.39%). After fine-tuning, performance improved substantially across all metrics, reaching 96.00% accuracy and a QWK of 0.988, indicating near-perfect agreement with the ground-truth labels.

The confusion matrix analysis showed that the baseline model struggled primarily with the middle score categories, where distinctions depend on subtle differences in prompt specificity and

completeness. Fine-tuning significantly reduced these errors and improved consistency across all score levels.

Although the final results are very strong, potential overfitting was carefully considered. Duplicate removal, dataset balancing, and leakage checks confirmed zero overlap between training, validation, and test sets, supporting the reliability of the evaluation. Nevertheless, future work should assess performance on external prompt collections to further evaluate generalization.

In addition to improved scoring accuracy, the fine-tuned model generated more rubric-aware rationales, making the system more interpretable and useful for educational and prompt-engineering applications.

6.2 Limitations

- The dataset is primarily synthetic and may not fully represent the diversity of real-world user prompts.
- Although the original candidate dataset contained 15,000 examples, the final experimental dataset used for training and evaluation was a smaller processed and balanced subset. While this improved data quality and consistency, it may have reduced dataset diversity.
- Some rationales were generated automatically and may still reflect synthetic writing patterns.
- The model was evaluated on prompt quality tasks similar to those used during dataset construction, which may limit generalization to completely unseen domains.
- The project focuses primarily on English-language prompt evaluation and does not extensively evaluate multilingual performance.

6.3 Reliability Considerations

Because synthetic datasets can contain repeated structures and patterns, deduplication and leakage detection were important components of the data preparation pipeline. Strict overlap checks confirmed zero overlap between the training, validation, and test sets, reducing the risk of data leakage and ensuring reliable evaluation. In addition, both the baseline and fine-tuned models

were evaluated on the same held-out test set, making the performance comparison fair and consistent.

The use of Quadratic Weighted Kappa (QWK) is particularly appropriate for this task because the score labels are ordinal rather than purely categorical. For example, predicting a score of 3 when the true score is 4 is less severe than predicting a score of 0 when the true score is 4. As a result, QWK provides a more informative measure of scoring agreement than accuracy alone.

7. Recommendations and Future Work

Although the proposed system achieved strong performance, several improvements can be explored in future work:

- **Multi-Model Comparison:** Evaluate additional instruction-tuned models such as Llama-3-8B-Instruct and Qwen-2.5-7B-Instruct to compare their effectiveness on rubric-based prompt evaluation.
- **Multilingual Support:** Extend the dataset and fine-tuning process to support Arabic and other languages, increasing the system's applicability across different educational and professional settings.
- **Dataset Expansion:** Incorporate more real-world prompts from diverse domains to improve generalization and reduce reliance on synthetic data.
- **Prompt Improvement Assistance:** Extend the system beyond evaluation by automatically suggesting improved versions of low-quality prompts.
- **Cross-Domain Evaluation:** Assess model performance on specialized domains such as software engineering, healthcare, and legal writing to better understand its generalization capabilities.

8.Web Application Implementation

A lightweight web application was developed using Streamlit to demonstrate the fine-tuned model. Users can enter a prompt, receive a predicted quality score (0–4), and view a generated rationale explaining the evaluation. Streamlit was chosen because it enables rapid deployment, seamless integration with Python-based machine learning models, and a simple user-friendly interface.

9. Team Contributions

Team Member	Contribution
Mayar Basheer	Fine-tuning implementation, dataset processing support, and experimental setup, Streamlit web app development,
Omar El Haj Daoud	Baseline inference, training pipeline support, and code integration.
Yosef Dabeek	Dataset design and refinement, evaluation analysis, report preparation, and result interpretation.

10. Conclusion

This project successfully developed a Prompt Quality Evaluator using an instruction-tuned large language model. The system evaluates submitted prompts according to a structured rubric and generates both a quality score and a supporting rationale. A custom dataset was collected, processed, balanced, and prepared for instruction tuning, and Mistral-7B-Instruct-v0.2 was evaluated before and after LoRA fine-tuning.

The experimental results demonstrated a substantial performance improvement after fine-tuning. Accuracy increased from 56.39% to 96.00%, MAE decreased from 0.5013 to 0.0425, and QWK improved from 0.8490 to 0.9880. These findings show that parameter-efficient fine-tuning can effectively adapt a general-purpose LLM to a specialized rubric-based evaluation task.

The project also highlights the importance of dataset quality, class balancing, leakage prevention, and appropriate evaluation metrics when developing reliable AI systems. Overall, the proposed Prompt Quality Evaluator provides a strong foundation for educational applications, prompt engineering assistance, and automated feedback systems.

10. References

- [1] Mistral AI, *Mistral-7B-Instruct-v0.2 Model Documentation*. Available: <https://docs.mistral.ai/>
- [2] Hugging Face, *Transformers Documentation*. Available: <https://huggingface.co/docs/transformers>
- [3] Unsloth AI, *Unsloth Documentation for Efficient LLM Fine-Tuning*. Available: <https://docs.unsloth.ai/>
- [4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations (ICLR)*, 2022.
- [5] Course Project Instructions, *Open-Ended Project: Build Your Own Automatic Evaluator with Instruction-Tuned Transformers*, An-Najah National University.
- [6] Scikit-learn Developers, *Scikit-learn Documentation*. Available: <https://scikit-learn.org/>